MODULE 1

PRACTICAL NO 1 : PERFORM GEOMETRIC TRANSFORMATION USING PYTHON

```
import numpy as np

import matplotlib.pyplot as plt

import matplotlib.transforms as transforms


# Original points

points = np.array([[1, 1], [2, 2], [3, 1]])


# Translation

translation_matrix = np.array([[1, 0, 2], [0, 1, 3], [0, 0, 1]])  # Translation by (2, 3)

translated_points = np.dot(translation_matrix, np.hstack([points, np.ones((points.shape[0],
1))]).T).T[:, :2]


# Rotation

theta = np.pi / 4  # Rotation angle (45 degrees)

rotation_matrix = np.array([[np.cos(theta), -np.sin(theta), 0], [np.sin(theta), np.cos(theta), 0], [0, 0,
1]])

rotated_points = np.dot(rotation_matrix, np.hstack([points, np.ones((points.shape[0], 1))]).T).T[:, :2]


# Scaling

scaling_matrix = np.array([[2, 0, 0], [0, 2, 0], [0, 0, 1]])  # Scaling by a factor of 2

scaled_points = np.dot(scaling_matrix, np.hstack([points, np.ones((points.shape[0], 1))]).T).T[:, :2]


# Plotting

plt.figure(figsize=(10, 5))


plt.subplot(1, 3, 1)

plt.title('Translation')

plt.plot(points[:, 0], points[:, 1], 'bo', label='Original')

plt.plot(translated_points[:, 0], translated_points[:, 1], 'r+', label='Translated')

plt.axis('equal')
```

```
plt.legend()
plt.subplot(1, 3, 2)
plt.title('Rotation')
plt.plot(points[:, 0], points[:, 1], 'bo', label='Original')
plt.plot(rotated_points[:, 0], rotated_points[:, 1], 'r+', label='Rotated')
plt.axis('equal')
plt.legend()
plt.subplot(1, 3, 3)
plt.title('Scaling')
plt.plot(points[:, 0], points[:, 1], 'bo', label='Original')
plt.plot(scaled_points[:, 0], scaled_points[:, 1], 'r+', label='Scaled')
plt.axis('equal')
plt.legend()
plt.show()
```

Output:

